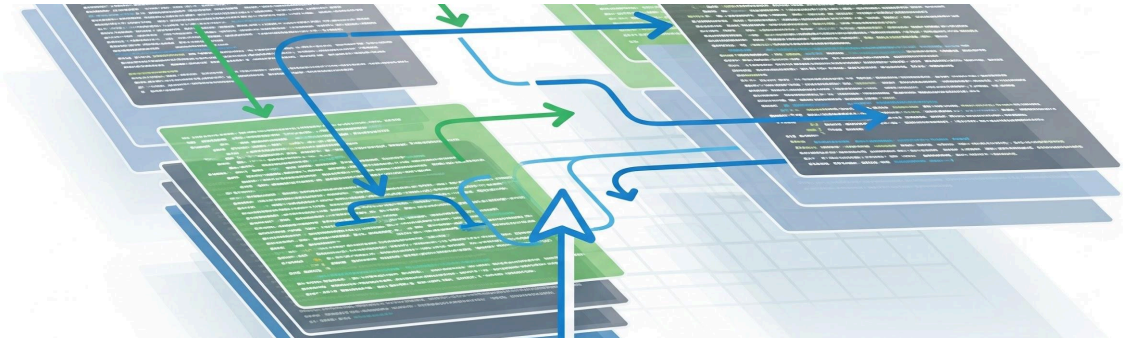

Git Complete Documentation (DevOps Day 1)

1. What is Git?

Git is a Version Control System (VCS) used to track changes in code.

Simple definition: Git = Tool to track, manage, and store code changes

Real-Time Example

	Without Git	With Git
History/Backup	File overwritten ❌ No backup ❌	Full history ✅
Collaboration	Team conflict ❌	Safe collaboration ✅
Rollback		Easy rollback ✅

2. Why Git?

Git solves the following common development issues:

- Code overwrite issues
- No version history
- Team conflicts

Example: Version 1 → Version 2 → Version 3. You can go back anytime ⏪

3. Git vs GitHub

This table compares the features of Git and GitHub.

Feature	Git	GitHub
Type	Tool	Platform
Location	Local	Cloud
Use	Track code	Store & share

Flow: Developer → Git → GitHub

4. Git Architecture

Working Directory → Staging Area → Repository

- **Working Directory:** where you edit code
- **Staging:** ready to save
- **Repository:** permanently saved

5. Key Concepts

- **Repository (📁):** Project folder tracked by Git
- **Commit (💾):** Snapshot of project.
    - Example: Commit 1 → Initial code; Commit 2 → Added feature; Commit 3 → Bug fix
- **Branch (🌿):** Separate environment
    - Example:
        - `main` → Production
        - `dev` → Testing
        - `feature` → Development

6. Git Installation (Linux)

Use the following commands to install Git on Linux:

`sudo dnf install git -y`
Check: `git --version`

7. Basic Commands (Hands-on)

- **Initialize repo (✅):** `git init`
- **Check status (✅):** `git status`
- **Add files (✅):** `git add .`
- **Commit (✅):** `git commit -m "first commit"`

8. Working with Remote (GitHub)

Clone Repository (🔽) - Git Clone

`git clone https://github.com/pravinaws02/Git-25032026.git`This command downloads the project from GitHub.

Git Push (🔼) - developer

`git push origin main`This command uploads code to GitHub.

Git Pull (🔽) - devops

`git pull origin main`This command downloads the latest code from GitHub.

How Pull Works: GitHub → `git pull` → Local system updated

Git Merge (🔄)

`git merge origin/branch name`
This command combines branches.

Git Commit - save

Git and GitHub Workshop Notes: Real-Time Workflow and Best Practices

9. Complete Real-Time Workflow 👨‍💻 Scenario

The following steps outline a standard real-time workflow for developing a feature.

1. **Clone the Repository:**
   `git clone repo_url`
2. **Navigate to the Repository Directory:**
   `cd repo`
3. **Create and Switch to a New Feature Branch:**
   `git checkout -b feature-login`
4. **Make Necessary Code Changes**
   `# Make changes`
5. **Stage Changes:**
   `git add .`
6. **Commit Changes with a Descriptive Message:**
   `git commit -m "Added login"`
7. **Push the New Branch to the Remote Repository:**
   `git push origin feature-login`
8. **Switch Back to the Main Branch:**
   `git checkout main`
9. **Merge the Feature Branch into Main:**
   `git merge feature-login`
10. **Push the Updated Main Branch to the Remote Repository:**
    `git push origin main`

10. Important Commands Summary

Command	Purpose
`git init`	Start repo
`git clone`	Download repo
`git status`	Check changes
`git add`	Stage files
`git commit`	Save changes
`git push`	Upload code

Command	Purpose
`git pull`	Get code
`git merge`	Combine branches

11. Common Errors (VERY IMPORTANT)

A summary of common errors encountered when using Git and their fixes.

Error	Reason	Fix
❌ Error 1: Not a Git Repository `fatal: not a git repository`	Not inside a repository directory.	`cd your-repo` OR `git clone repo_url`
❌ Error 2: Wrong Folder (Your case) 👉 Example: `cd Git-12032026`	Navigating to the incorrect local repository folder.	Correct: `cd Git-25032026` ✅
❌ Error 3: File with spaces `cat OTP lambda`	The filename contains spaces and wasn't quoted.	✔ Fix: `cat "OTP lambda"`
❌ Error 4: Authentication Failed	The user is providing an incorrect password or token.	👉 Use: - Username - Personal Access Token (not password)

12. Debug Commands

These commands are useful for debugging and checking the state of your current environment and repository.

- `pwd`: Print working directory (check your current folder)
- `ls`: List directory contents
- `ls -a`: List all directory contents, including hidden files (useful to check for the hidden `.git` folder)
- `git status`: Check the status of your working tree
- `git remote -v`: Check the remote repositories configured for your local repository

13. Best Practices

Adhering to these best practices helps maintain a clean and effective version control system.

- ✅ Always pull before starting work
- ✅ Use branches for all new features and fixes
- ✅ Write proper, descriptive commit messages
- ✅ Don't work directly on `main` (or `master`)

14. Classroom Demo Script 🎯

Step-by-step live demonstration for a quick commit and push:

1. **Clone the Demo Repository:**
   `git clone https://github.com/pravinaws02/Git-25032026.git`
2. **Navigate into the Repository:**
   `cd Git-25032026`
3. **Create a New File:**
   `touch file1.txt`
4. **Stage the New File:**
   `git add .`
5. **Commit the Change:**
   `git commit -m "added file"`
6. **Push to the Remote Main Branch:**
   `git push origin main`